DigiM — Commercial Architecture & Edge Cases Specification

This document provides a comprehensive operational and architectural analysis of **DigiM** as it transitions from a single-brand prototype to a **commercial, multi-tenant SaaS application**. It outlines known edge cases, failure modes, security boundaries, and recommended mitigations across all core subsystems.

---

1. Authentication, Identity & Multi-Tenancy

1.1 Tenant ID Security & Server-Side Authority

* **Current State**: The frontend reads `tenant_id` from browser `localStorage` and appends it as a query parameter (e.g., `?tenant_id=1`) to backend requests.

* **Edge Case (Client Tampering)**: A user authenticated as Account A could manually edit `localStorage.setItem('tenant_id', '2')` in their browser console or send forged requests with `?tenant_id=2` to view or mutate another business's campaigns, brand assets, or Facebook tokens.

* **Commercial Mitigation (P0)**:

* Eliminate client-supplied `tenant_id` query parameters entirely.

* The backend must validate the Google JWT session token in `Authorization: Bearer` on every API request.

* The authenticated user's `tenant_id` must be extracted strictly from the verified database record on the server:

```python

def get_current_tenant(

token_payload: dict = Depends(verify_google_sso_token),

db: Session = Depends(get_db)

) -> Tenant:

tenant = db.query(Tenant).filter_by(google_sso_id=token_payload["sub"]).first()

if not tenant:

raise HTTPException(status_code=401, detail="Unauthorized")
```

return tenant

```

### 1.2 Team Collaboration (Multiple Users per Business)

* **Current State**: Each distinct Google email dynamically creates its own isolated `Tenant`.

* **Edge Case (Colleague / Agency Access)**: If the business owner (`owner@brand.com`) connects Facebook and creates campaigns, and their marketing assistant (`assistant@brand.com`) logs in with Google, the assistant is currently placed in a brand-new, empty workspace rather than sharing the business workspace.

* **Commercial Mitigation (P1)**:

* Leverage the existing `users` table (`tenant_id`, `email`, `role`).

* Introduce an **"Invite Team Member"** workflow:

1. Owner invites `assistant@brand.com` with role `editor`.

2. An entry is added to `users` linked to `tenant_id`.

3. When `assistant@brand.com` logs in via Google SSO, the auth handler checks the `users` table first. If invited, it associates their session with that existing `tenant_id`.

### 1.3 Google ID vs Email Changes & Aliases

* **Edge Case**: A business user updates their primary email address in Google Workspace or uses an email alias (`owner@brand.com` vs `marketing@brand.com`).

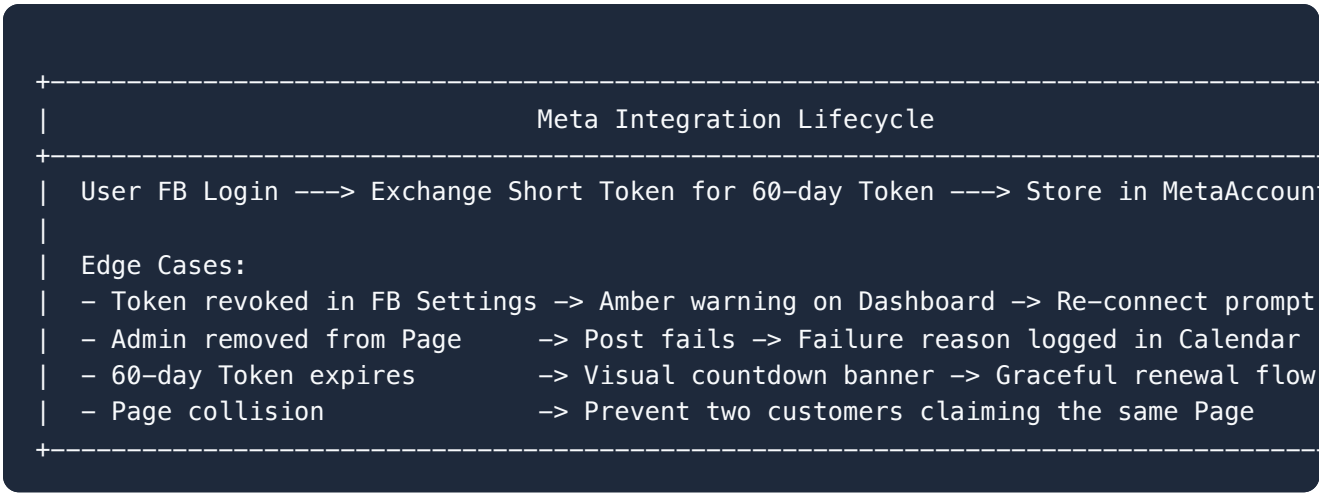
* **Mitigation**:

| | |
|---|---|
| **\* The backend matches accounts using `(Tenant.email == email)** | **(Tenant.google_sso_id == google_id)`.** |

* Google's `sub` claim (`google_sso_id`) is globally unique and immutable, ensuring users never lose access to their workspace even if their email alias changes.

---
```

2. Meta (Facebook & Instagram) Integration



2.1 Token Revocation from Facebook's Side

* **Edge Case**: A user changes their personal Facebook password or navigates to `Facebook -> Settings -> Business Integrations` and revokes access to DigiM.

* **Failure Mode**: All subsequent API calls fail with Meta OAuth error `code 190, subcode 458 (User revoked authorization)`.

* **Mitigation**:

* The `/api/meta/status` health check queries the Meta Graph debug endpoint (`/debug_token`).

* When invalid, the response returns `connected: true, token_valid: false`.

The frontend renders an amber warning banner: "Session expired — click Re-connect to restore auto-publishing"*.

2.2 Permissions Dropped on the Facebook Page

* **Edge Case**: The Facebook account that authorized DigiM is demoted from "Page Admin" or "Task Manager" to a lower role, or removed from the Page entirely.

* **Failure Mode**: The token remains valid for the user, but publishing to the page returns Meta error `code 200 (Permissions error: User does not have sufficient permissions to perform this action)`.

* **Mitigation**:

* The scheduler captures Meta's exact error message.

* The campaign status changes from `scheduled` to `failed`.

* The error is saved in `campaigns.failure_reason` and surfaced directly on the Calendar card so the business owner knows exactly why the post did not go live.

2.3 Page Collision Prevention (Agencies & Shared Pages)

* **Edge Case**: Two independent DigiM customers (e.g., an agency and a local branch) attempt to connect the exact same Facebook Page ID (`page_id`).

* **Commercial Mitigation (P1)**:

* When `POST /api/meta/connect` is called, check whether `page_id` is already connected to another active `tenant_id` .

* If already connected to another tenant, require explicit confirmation or transfer verification before re-assigning publishing authority.

2.4 60-Day Token Expiration & Renewal

* **Current State**: Long-lived Facebook Page tokens expire after 60 days.

* **Edge Case**: A post scheduled 65 days in advance will fail if the token expires before the publish timestamp.

* **Mitigation**:

* Dashboard displays a live days-remaining counter when fewer than 14 days remain.

* When a user visits DigiM within the valid window, the backend can silently call the Meta token refresh endpoint to exchange the active token for a fresh 60-day token without requiring the user to re-authenticate manually.

---

3. Session, Browser & Client-Side Edge Cases

3.1 Shared Workstations & Device Switching

* **Edge Case**: A social media manager uses a single laptop to manage multiple clients, logging out of Client A and logging into Client B.

* **Failure Mode**: If `localStorage` retains state from Client A, Client B's session could display cached brand assets, draft campaigns, or company logos.

* **Mitigation (Implemented in `645f6b4`)**:

* On logout, DigiM explicitly cleanses all user and workspace data:

```javascript
localStorage.removeItem('user_logged_in');
localStorage.removeItem('tenant_id');
localStorage.removeItem('user_email');
```

localStorage.removeItem('user_name');

localStorage.removeItem('onboarding_completed');

localStorage.removeItem('businessProfile');

```

### 3.2 Third-Party Cookie Blocking (Safari ITP & Privacy Sandbox)

* **Edge Case**: Safari's Intelligent Tracking Prevention (ITP) and Chrome's third-party cookie restrictions frequently suppress Google One Tap floating iframes.

* **Mitigation**:

* DigiM implements dual authentication paths: Google One Tap for seamless instant login, with a standard Google Sign-In button ( ) permanently visible as a resilient fallback.

---

## 4. Cloud Infrastructure, Scaling & Database Architecture

| Dimension | Current Architecture (Prototype) | Commercial Target (Production SaaS) |
| --- | --- | --- |
| **Database** | SQLite file synced to Google Cloud Storage (GCS) on startup/shutdown | Managed Google Cloud SQL (PostgreSQL) |
| **Container Instances** | Single Cloud Run container (`--max-instances 1`) | Multi-instance auto-scaling (`--min-instances 1 --max-instances 10+`) |
| **Background Scheduler** | Python in-memory daemon thread (`process_scheduled_campaigns`) | Google Cloud Tasks / Cloud Scheduler |
| **Media Assets** | Local `/uploads` mirrored to GCS bucket | Direct signed uploads to GCS with Cloud CDN caching |

### 4.1 SQLite File Locking Under Multi-Instance Cloud Run

* **The Vulnerability**:

* Currently, the backend restores `marketflow.db` from GCS on startup and flushes back to GCS on teardown.

* If Cloud Run scales to **2 or more concurrent container instances** during peak traffic, both instances will write to their own local SQLite copies. The last instance to exit will overwrite the other's database file on GCS, causing **data loss**.
```

* **Commercial Mitigation (P0)**:

* Point `DATABASE_URL` to a Google Cloud SQL (PostgreSQL) instance.

* The backend SQLAlchemy models (`backend/db/schema.py`) and FastAPI endpoints are already 100% PostgreSQL-compatible; no code rewrite is required.

4.2 Distributed Background Scheduling

* **The Vulnerability**:

* In a multi-container environment, each container runs its own background polling thread. If two instances check `status == 'scheduled'` at the exact same minute, both instances might attempt to publish the **same post twice** to Facebook.

* **Commercial Mitigation (P1)**:

* Use atomic database row-locking (`SELECT FOR UPDATE SKIP LOCKED` in PostgreSQL) when selecting campaigns due for publication, ensuring only one worker instance can claim a post.

* Alternatively, trigger individual post jobs via **Google Cloud Tasks** at the scheduled timestamp.

4.3 Timezone-Aware Publishing Across Global Customers

* **Edge Case**: Customer A is in Bangalore (IST, UTC+5:30) and schedules a post for 10:00 AM. Customer B is in California (PST, UTC-8:00) and schedules a post for 10:00 AM.

* **Current Mitigation**:

* All `scheduled_time` values are converted to and stored in **UTC**.

* The background worker evaluates `datetime.datetime.utcnow()` against UTC timestamps.

* Timezone names (e.g. `Asia/Kolkata` , `America/Los_Angeles`) are inferred from target locations during onboarding and stored in `tenants.timezone` for accurate display in the Calendar UI.

---

5. Commercial Readiness Checklist

Critical Priority (P0 — Must complete before public paying customers)

- [x] **Remove Hardcoded Defaults**: Eliminated placeholder names (`Ravi Kumar`) and ensured profile renders dynamically from Google account credentials.
- [x] **Dynamic Account Resolution**: Replaced brittle seed tenant logic with dynamic Google SSO email/ID mapping.
- [x] **Active Token Sync Migration**: Resolved cross-tenant session drift on Cloud Run.

- [ ] **Switch to Cloud SQL (PostgreSQL)**: Transition from SQLite/GCS sync to managed Cloud SQL to enable safe multi-container auto-scaling.
- [ ] **Server-Side Token Verification**: Remove `?tenant_id=` from client queries and enforce tenant lookup via Google JWT in backend middleware.

High Priority (P1 — Next feature release)

- [ ] **Team Invitations (Multi-User Workspaces)**: Support inviting additional team members (`users` table) to a business workspace.
- [ ] **Cross-Tenant Page Collision Guard**: Prevent two independent customers from binding the same Facebook Page without transfer confirmation.
- [ ] **Automatic Token Refresh Worker**: Automatically renew 60-day Meta long-lived tokens in the background when valid tokens are within 14 days of expiration.
- [ ] **Distributed Job Scheduler**: Implement row-level locking (`SELECT ... FOR UPDATE`) or Cloud Tasks for distributed post publishing.

Medium Priority (P2 — Quality of Life & Compliance)

- [ ] **Export Data / GDPR Account Deletion**: Self-service endpoint to delete all tenant data, Facebook tokens, media assets, and campaigns.
- [ ] **Granular Audit Logs**: Expand `audit_logs` table to track all Facebook publish attempts, disconnects, and credential changes.